Subject: High Performance Computer Networks **Semester: 7th**

Assignment: MQTT Protocol

Batch no: 08

Project Name: Smart Environmental Monitoring System Using DHT22 and LED Control

Team Members:

Sl No	USN	Student Name	Semester/ Section
1.	1GA22EC033	CHARAN MAHENDARAN	7th A sec
2.	1GA22EC004	ADITYA NIMBARGI	7th A sec
3.	1GA22EC074	LIKHITH K	7th A sec

Objectives:

To measure temperature and humidity using the DHT22 sensor and send the data to an MQTT broker through the ESP32 via Wi-Fi for IoT-based monitoring.

Temperature and Humidity Monitoring

System Code

```
#include <WiFi.h>

#include <PubSubClient.h>

#include <DHTesp.h>

const int DHT_PIN = 15;

DHTesp dht;

const char* ssid = "Wokwi-GUEST"; ///  wifi ssid

const char* password = "";

const char* mqtt_server = "test.mosquitto.org";// mosquitto server url

WiFiClient espClient;

PubSubClient client(espClient);
```

```
unsigned long lastMsg = 0;
float temp = 0;
float hum = 0;
void setup_wifi() {
delay(10);
Serial.println();
Serial.print("Connecting to ");
Serial.println(ssid);
WiFi.mode(WIFI_STA);
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
delay(500);
Serial.print(".");
}
randomSeed(micros());
Serial.println("");
Serial.println("WiFi connected");
Serial.println("IP address: ");
Serial.println(WiFi.localIP());
}
void callback(char* topic, byte* payload, unsigned int length) {
Serial.print("Message arrived [");
Serial.print(topic);
Serial.print("] ");
for (int i = 0; i < length; i++) {
Serial.print((char)payload[i]);
}}
```

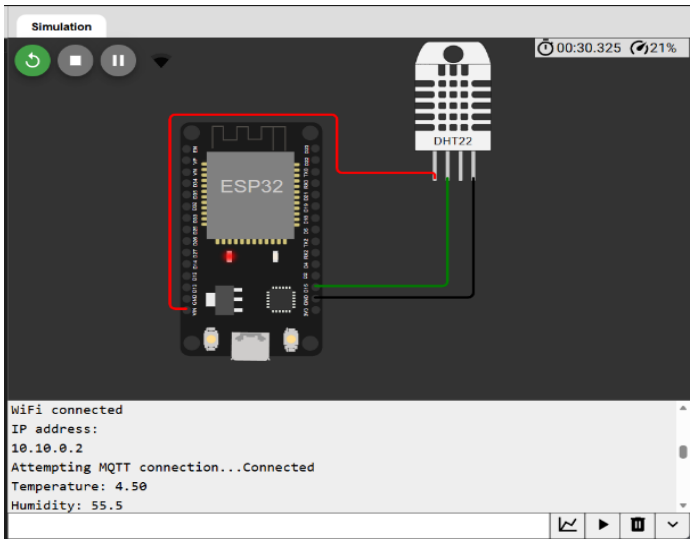
```
void reconnect() {
while (!client.connected()) {
Serial.print("Attempting MQTT connection...");
String clientId = "ESP32Client-";
clientId += String(random(0xffff), HEX);
if (client.connect(clientId.c_str())) {
Serial.println("Connected");
client.publish("/ThinkIOT/Publish", "Welcome");
client.subscribe("/ThinkIOT/Subscribe");
} else {
Serial.print("failed, rc=");
Serial.print(client.state());
Serial.println(" try again in 5 seconds");
delay(5000);
}}
}
void setup() {
pinMode(2, OUTPUT);
Serial.begin(115200);
setup_wifi();
client.setServer(mqtt_server, 1883);
client.setCallback(callback);
dht.setup(DHT_PIN, DHTesp::DHT22);
}
void loop() {
if (!client.connected()) {
reconnect();
```

```
}
client.loop();
unsigned long now = millis();
if (now - lastMsg > 2000) { //perintah publish data
lastMsg = now;
TempAndHumidity  data = dht.getTempAndHumidity();
String temp = String(data.temperature, 2);
client.publish("/ThinkIOT/temp", temp.c_str()); // publish temp topic /ThinkIOT/temp
String hum = String(data.humidity, 1);
client.publish("/ThinkIOT/hum", hum.c_str());   // publish hum topic /ThinkIOT/hum
Serial.print("Temperature: ");
Serial.println(temp);
Serial.print("Humidity: ");
Serial.println(hum);}}
```

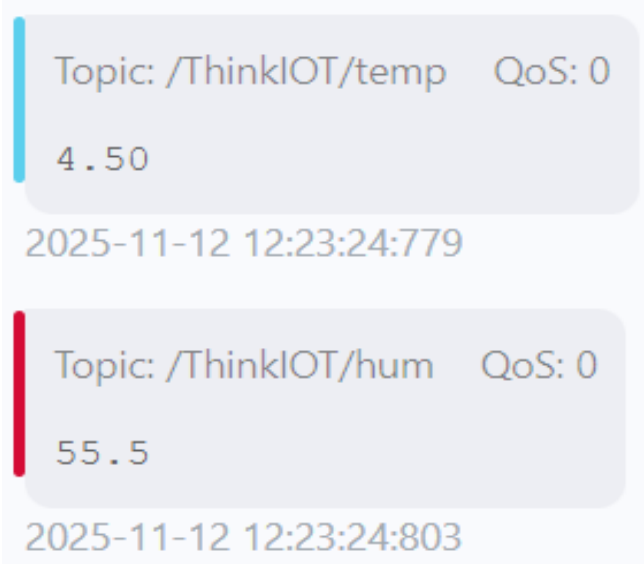
Explanation

The provided code connects an ESP32 microcontroller to a Wi-Fi network and an MQTT broker to create an IoT-based temperature and humidity monitoring system. The DHT22 sensor is connected to the ESP32 to measure temperature and humidity values, which are read using the DHT esp library. After connecting to the Wi-Fi network and the MQTT server the ESP32 continuously reads sensor data every two seconds. It then publishes the temperature and humidity readings to specific MQTT topics allowing other devices or applications subscribed to these topics to receive the data in real time. The code also includes a callback function to handle incoming messages and a reconnect function to ensure the MQTT connection is automatically restored if lost. Overall, this program demonstrates how ESP32 can be used for wireless environmental data collection and cloud-based IoT communication.

Results



The image shows a simulation of an IoT-based temperature and humidity monitoring system using an ESP32 microcontroller and a DHT22 sensor. The DHT22 sensor is connected to the ESP32 with three wires — the red wire provides a 3.3V power supply, the black wire is connected to the ground (GND), and the green wire connects the sensor's data pin to GPIO 15 of the ESP32. The serial monitor output at the bottom confirms that the ESP32 successfully connected to the Wi-Fi network and the MQTT broker, displaying an assigned IP address. It also shows the sensor readings: a temperature of 4.50°C and humidity of 55.5%. This verifies that the ESP32 is correctly reading data from the DHT22 and transmitting it through MQTT for IoT communication.



The image shows the temperature and humidity data received in MQTTx, which is used as the MQTT client to monitor published messages. The ESP32 sends the sensor readings to the MQTT broker under the topics /ThinkIOT/temp and /ThinkIOT/hum. MQTTx then subscribes to these topics and displays the values 4.50°C for temperature and 55.5% for humidity confirming successful data transmission and real-time IoT communication.

IoT-Based LED Control Using ESP32 and MQTT Protocol

System Code

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <ctype.h> //
#define LED_PIN 26 //
const char* ssid = "Wokwi-GUEST";
const char* password = "";
const char* mqttServer = "broker.hivemq.com";
const char* mqttClientID = "my-led-controller-final";
const char* ledTopic = "home/led";
WiFiClient espClient;
```

```
PubSubClient client(espClient);
void callback(char* topic, byte* payload, unsigned int length) {
Serial.print("Message arrived [");
Serial.print(topic);
Serial.print("] ");
char cleanPayload[64];
int cleanIndex = 0;
for (int i = 0; i < length && cleanIndex < 63; i++) {
char c = (char)payload[i];
if (!isspace(c)) {
cleanPayload[cleanIndex] = toupper(c);
cleanIndex++;}}
cleanPayload[cleanIndex] = '\0';
Serial.print(cleanPayload);
Serial.println("");
if (strcmp(topic, ledTopic) == 0)
if (strcmp(cleanPayload, "ON") == 0) {
digitalWrite(LED_PIN, HIGH);
Serial.println("-> LED turned ON");}
else if (strcmp(cleanPayload, "OFF") == 0) {
digitalWrite(LED_PIN, LOW);
Serial.println("-> LED turned OFF");}}}
void setup_wifi() {
delay(10);
Serial.println();
Serial.print("Connecting to ");
Serial.println(ssid);
```

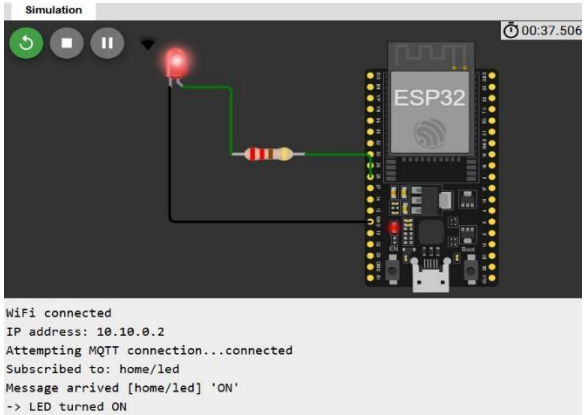
```
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
delay(500);
Serial.print(".");}
Serial.println("\nWiFi connected");
Serial.print("IP address: ");
Serial.println(WiFi.localIP());
}
void reconnect() {
while (!client.connected()) {
Serial.print("Attempting MQTT connection...");
if (client.connect(mqttClientID)) {
Serial.println("connected");
// Subscribe to the LED topic
client.subscribe(ledTopic);
Serial.print("Subscribed to: ");
Serial.println(ledTopic);
} else {
Serial.print("failed, rc=");
Serial.print(client.state());
Serial.println(" try again in 5 seconds");
delay(5000);}}}
void setup() {
Serial.begin(115200);
pinMode(LED_PIN, OUTPUT);
// Ensure LED starts OFF
digitalWrite(LED_PIN, LOW);
```

```
setup_wifi();
client.setServer(mqttServer, 1883);
client.setCallback(callback);
}
void loop() {
if (!client.connected()) {
reconnect();
}
client.loop();
}
```

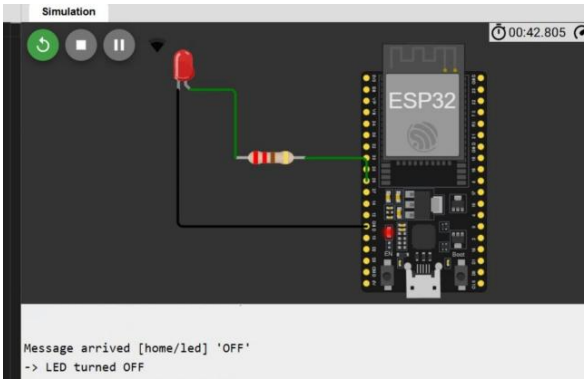
Explanation

The program enables remote and wireless operation of an LED using internet-based communication. It first connects the microcontroller to a Wi-Fi network, allowing it to access the internet. Once online, it establishes a link with a remote server that acts as a message exchange platform between the user and the device. When a user sends a command such as "ON" or "OFF" from another device connected to the same server, the system receives this instruction, interprets it, and performs the corresponding action by turning the LED either on or off. The system continuously monitors for new incoming commands, ensuring that any change made by the user is immediately executed in real time. Additionally, it includes an automatic reconnection feature, so if the internet or server connection is temporarily lost, the system will attempt to reconnect and resume normal operation without manual intervention. Overall, this setup demonstrates how simple electronic devices can be controlled wirelessly through cloud-based communication, forming the foundation of basic Internet of Things (IoT) applications.

Results



The image shows a simulation of an IoT-based LED control system using an ESP32 microcontroller connected to an external LED through a current-limiting resistor. The ESP32 establishes a Wi-Fi connection and successfully connects to an MQTT broker, enabling remote message-based device control. As indicated in the serial monitor, the system subscribes to a specific topic and receives the command "ON" from a remote MQTT client. In response, the LED connected to the ESP32 turns on, visually confirming successful command execution. The glowing LED signifies that the ESP32 correctly received and processed the message transmitted over the internet, thereby activating the output device. This demonstrates real-time wireless control of a physical component through cloud-based communication, representing a fundamental concept in IoT applications.



The image depicts the same IoT-based LED control setup in which the LED is connected to the ESP32 microcontroller via a resistor for safe operation. After establishing Wi-Fi and MQTT

connections, the ESP32 remains subscribed to the control topic and continuously listens for incoming commands. When the MQTT client sends the message "OFF", the system receives and interprets it, resulting in the LED being turned off, as seen in the simulation. The serial monitor output confirms successful message reception and acknowledges the command execution with the statement "LED turned OFF". This verifies reliable communication between the remote user interface and the ESP32, showing how IoT-based control enables efficient and wireless operation of electronic devices through cloud messaging.

3. Conclusion

The IoT-based temperature and humidity monitoring system using the DHT22 sensor and ESP32 successfully demonstrates real-time environmental data acquisition and wireless communication through the MQTT protocol. By connecting the ESP32 to a Wi-Fi network and publishing sensor readings to specific MQTT topics, the system efficiently transmits temperature and humidity data to the cloud for remote monitoring. The results verify that the ESP32 accurately reads and publishes the DHT22 sensor outputs, while MQTT ensures seamless data delivery to subscribed clients such as MQTTx. This implementation highlights the potential of IoT in smart environmental monitoring, where real-time data sharing enables efficient decision-making and automation in various applications like agriculture, weather observation, and smart homes.

Similarly, the IoT-based LED control system using the ESP32 and MQTT protocol demonstrates the effectiveness of wireless remote operation through cloud-based communication. The ESP32 connects to the internet and responds to commands such as "ON" and "OFF" received from an MQTT client, controlling the LED state accordingly. The successful reception and execution of these commands confirm reliable two-way communication between the device and the user over the network. This project effectively illustrates how basic electronic components can be integrated into IoT frameworks to achieve automation and remote control, forming a foundation for more advanced smart systems involving interactive device management and real-time response.